

1. Introdução

Com o avanço da tecnologia, a utilização de sistemas de informatização vem crescendo, usando as aplicações web e os serviços digitais, tornando a segurança da informação um aspecto fundamental para as organizações que trabalham com tais elementos. Empresas dos mais diferentes setores armazenam e conseqüentemente processam informações importantes de forma diária, como os dados pessoais de clientes, as informações financeiras, as credenciais de acesso e os dados internos. Assim, falhas de segurança em sistemas podem gerar problemas e conseqüências significativas, incluindo o vazamento de informações, conseqüentemente prejuízos financeiros, gerando a interrupção de serviços e danos à imagem da organização.

Nesse contexto, destaca-se a OWASP (Open Worldwide Application Security Project), uma organização que atua na promoção de conhecimentos, boas práticas e ferramentas voltadas à segurança de aplicações. Um de seus principais trabalhos é o OWASP Top 10, um modelo que apresenta e organiza alguns dos principais riscos relacionados à segurança de aplicações web. O objetivo de tal modelo é auxiliar os desenvolvedores, os profissionais de tecnologia e os demais responsáveis pela segurança das organizações, com o intuito de compreender os riscos mais relevantes e, assim, adotar medidas para a redução das possibilidades de exploração de vulnerabilidades.

O OWASP Top 10 apresenta dez categorias de riscos de segurança que devem receber atenção durante o desenvolvimento e a manutenção de aplicações. Entre elas estão: A01 – Broken Access Control (Controle de Acesso Quebrado); A02 – Security Misconfiguration (Falhas de Configuração de Segurança); A03 – Injection (Injeção); A04 – Insecure Design (Projeto Inseguro); A05 – Security Misconfiguration (Configuração Incorreta de Segurança); A06 – Vulnerable and Outdated Components (Componentes Vulneráveis e Desatualizados); A07 – Identification and Authentication Failures (Falhas de Identificação e Autenticação); A08 – Software and Data Integrity Failures (Falhas de Integridade de Software e Dados); A09 – Security Logging and Monitoring Failures (Falhas de Registro e Monitoramento de Segurança); e A10 – Server-Side Request Forgery – SSRF (Falsificação de Solicitação do Lado do Servidor).

A utilização de tal modelo é de extrema importância, porque permite que as empresas tenham uma certa referência para identificar e, assim, avaliar os riscos existentes em seus sistemas. Entretanto, a segurança não deve ser levada em consideração apenas após o desenvolvimento de uma aplicação ou quando ocorre um incidente. As práticas de segurança devem ser usadas durante todo o projeto, durante todo o ciclo de desenvolvimento, desde o planejamento até a implementação, fazendo uso de testes para verificação e implantação, além da manutenção do sistema.

Dessa maneira, conhecer as ameaças apresentadas pelo OWASP Top 10 possibilita que as organizações adotem uma postura preventiva em relação à segurança. A identificação antecipada de vulnerabilidades, associada às boas práticas de desenvolvimento seguro, podem reduzir significativamente os riscos de comprometimento dos sistemas e das informações. Assim, este trabalho irá utilizar o modelo OWASP como referência para analisar duas ameaças de segurança, apresentar exemplos reais de exploração dessas

vulnerabilidades e propor contramedidas que poderiam ser adotadas pela empresa XPTO para reduzir sua exposição a ataques.

2. Objetivo

O presente trabalho tem como objetivo analisar a importância do desenvolvimento seguro de sistemas, utilizando como referência o modelo OWASP Top 10, que apresenta as principais categorias de riscos relacionados à segurança de aplicações. Assim, busca-se compreender como as vulnerabilidades podem ser exploradas por agentes mal-intencionados e quais os impactos de um ataque para uma organização.

Para a realização da análise, serão abordadas duas ameaças de segurança: Broken Access Control (Controle de Acesso Quebrado) e Injection (Injeção). A escolha dessas ameaças está relacionada à importância dos mecanismos de controle de acesso e de validação das informações feitas de forma adequada para a proteção dos sistemas e dos dados corporativos.

Na primeira parte da análise, será abordada a ameaça de Broken Access Control, apresentando o seu conceito, as possíveis formas de exploração e os impactos que uma falha desse tipo poderia causar à empresa XPTO. Também será apresentado um caso real de comprometimento relacionado a tal categoria, acompanhado de recomendações de segurança que podem ser utilizadas, visando a redução de riscos.

Em seguida, será analisada a ameaça de Injection, também considerando seus principais conceitos, as possíveis consequências para a organização e um exemplo real de exploração. Também serão apresentadas contramedidas, juntamente com boas práticas que podem ser implementadas pela empresa XPTO para prevenir ou reduzir a possibilidade desse tipo de ataque.

Dessa forma, o trabalho busca relacionar os conceitos apresentados pelo modelo OWASP com situações práticas de segurança, permitindo identificar os riscos, compreender os seus impactos e propor as medidas preventivas que contribuam para o desenvolvimento e a manutenção de sistemas mais seguros.

3. AMEAÇA 1 – BROKEN ACCESS CONTROL

3.1 Tipo de ameaça – A01:2021 Broken Access Control

O Broken Access Control, também denominado Controle de Acesso Quebrado, corresponde a falhas nos mecanismos responsáveis por determinar quais os recursos e funcionalidades que cada usuário de uma aplicação está autorizado a acessar. De acordo com a OWASP, o controle de acesso visa garantir que os usuários não consigam realizar ações além das permissões que eles possuem. Quando esses controles não são implementados corretamente ou deixam de ser implementados, um usuário pode conseguir acessar informações, funcionalidades ou recursos que, em teoria, ele não poderia.

Na versão 2021 do OWASP Top 10, o Broken Access Control ocupa a posição A01, sendo classificado como a principal categoria de risco para aplicações web. A própria OWASP informa que essa categoria passou da quinta posição na edição anterior para a primeira posição em 2021, devido à quantidade de ocorrências identificadas nos dados analisados.

Esse tipo de vulnerabilidade pode ocorrer de diferentes maneiras. Um exemplo é permitir que o usuário comum altere uma informação pertencente a outro usuário, simplesmente modificando um identificador enviado na requisição. Outra possibilidade seria um usuário, não administrador, conseguir acessar diretamente a função destinada exclusivamente aos administradores. Também podem ocorrer falhas quando determinadas operações de uma API não possuem controles de autorização adequados ou até mesmo quando os mecanismos utilizados para representar permissões são manipulados.

3.2 Possíveis impactos para a empresa XPTO

Uma falha do controle de acesso pode representar um risco significativo, pois pode permitir que os usuários não autorizados consigam acesso a informações ou funcionalidades que deveriam estar protegidas. Dependendo da natureza da aplicação e dos privilégios obtidos, um atacante poderia visualizar, modificar ou até mesmo excluir informações pertencentes a outros usuários.

Entre os possíveis impactos estão o acesso não autorizado a informações confidenciais, alterando ou destruindo dados, elevando indevidamente os privilégios e a execução de funcionalidades que deveriam estar disponíveis somente para determinados usuários. Em um cenário corporativo, as consequências podem comprometer informações de clientes, dados internos, registros financeiros ou outros recursos que são importantes para o funcionamento da organização.

Além dos impactos técnicos, o comprometimento de sistemas também podem gerar consequências administrativas e financeiras para a empresa. O acesso indevido a informações pode resultar em perda de confiança por parte dos clientes e parceiros, interrompendo assim as atividades e causando danos à imagem da empresa. Portanto, o controle adequado das permissões deve ser considerado uma parte fundamental do desenvolvimento seguro de sistemas.

Dessa forma, a empresa XPTO deve garantir que cada usuário tenha acesso somente aos recursos necessários para desempenhar suas funções, aplicando controles de autorização no lado do servidor e realizando testes para verificar se usuários comuns não conseguem acessar funcionalidades ou dados destinados a usuários com maiores privilégios.

4. EXEMPLO REAL – AMEAÇA 1

4.1 Caso Facebook – Vulnerabilidade na função “View As”

Um exemplo real relacionado a falhas de controle de acesso ocorreu no Facebook. O incidente envolveu uma vulnerabilidade presente na funcionalidade denominada “View As”, utilizada para permitir que um usuário visualize como seu perfil aparecia para outras pessoas.

Em setembro de 2018, o Facebook identificou que invasores estavam explorando uma vulnerabilidade relacionada a essa funcionalidade. Inicialmente, a empresa informou que aproximadamente 50 milhões de contas poderiam ter sido afetadas. Posteriormente, após uma investigação mais detalhada, o Facebook informou que cerca de 30 milhões de contas tiveram seus tokens de acesso efetivamente roubados.

O problema ocorreu devido à combinação de três falhas diferentes no funcionamento do sistema. A primeira fazia com que a funcionalidade “View As”, que deveria ser apenas uma ferramenta de visualização, apresentasse indevidamente uma função de publicação de vídeos. A segunda falha fazia com que o carregador de vídeos gerasse incorretamente um token de acesso com permissões associadas ao aplicativo móvel do Facebook. A terceira falha fazia com que esse token fosse gerado para a pessoa cujo perfil estava sendo visualizado, em vez de ser gerado para o usuário que estava realizando a visualização.

Assim, os invasores exploraram essa combinação de falhas, obtendo tokens de acesso pertencentes a outras contas. Esses tokens funcionavam como uma espécie de chave digital, utilizada para manter os usuários autenticados na plataforma. Com os tokens obtidos, os atacantes puderam assumir o controle das contas afetadas e utilizar o acesso para avançar para outras contas.

Aproximadamente 15 milhões de pessoas tiveram informações como nome e dados de contato acessados. Outras 14 milhões tiveram, além dessas informações, diversos outros dados presentes em seus perfis potencialmente acessados, incluindo nome de usuário, gênero, idioma, relacionamento, cidade, data de nascimento, dispositivos utilizados, educação, trabalho e pesquisas recentes. Para aproximadamente 1 milhão de pessoas, o Facebook informou que os invasores não acessaram as informações.

4.2 Relação do incidente com a ameaça estudada

O caso demonstra a importância de implementar corretamente os mecanismos de autorização e controle de acesso. A funcionalidade “View As” deveria permitir somente a visualização de um perfil, mas a combinação de falhas fez com que uma ação aparentemente limitada pudesse resultar na geração de credenciais de acesso associadas a outro usuário.

Em um cenário semelhante na empresa XPTO, uma falha desse tipo poderia permitir que certo usuário obtivesse acesso aos recursos ou as informações pertencentes a outra pessoa, ultrapassando os limites de suas permissões. Dependendo dos privilégios associados à conta comprometida, o atacante poderia obter acesso a informações confidenciais, alterar dados ou utilizar a conta comprometida para alcançar outros recursos do sistema.

O incidente também demonstra que uma vulnerabilidade de controle de acesso não necessariamente ocorre devido a uma única falha isolada. A interação entre diferentes funcionalidades e mecanismos de autenticação e autorização pode criar uma situação que permita a um atacante ultrapassar os controles de segurança previstos pela aplicação.

4.3 Medidas adotadas após o incidente

Após identificar o ataque, o Facebook corrigiu a vulnerabilidade e desativou temporariamente a funcionalidade “View As” enquanto realizava uma revisão de segurança. A empresa também invalidou os tokens de acesso das contas potencialmente afetadas. Inicialmente, foram redefinidos os tokens de aproximadamente 50 milhões de contas diretamente afetadas e, por precaução, outros 40 milhões de contas que haviam sido alvo de consultas utilizando a funcionalidade “View As”.

O caso demonstra, portanto, como uma falha na implementação de controles de acesso e na interação entre diferentes componentes de uma aplicação podem resultar em um comprometimento de grande escala. Esse incidente serve como exemplo da necessidade de validar as permissões no lado do servidor, limitar os privilégios dos usuários, testar diferentes níveis de acesso e monitorar comportamentos anormais dentro da aplicação.

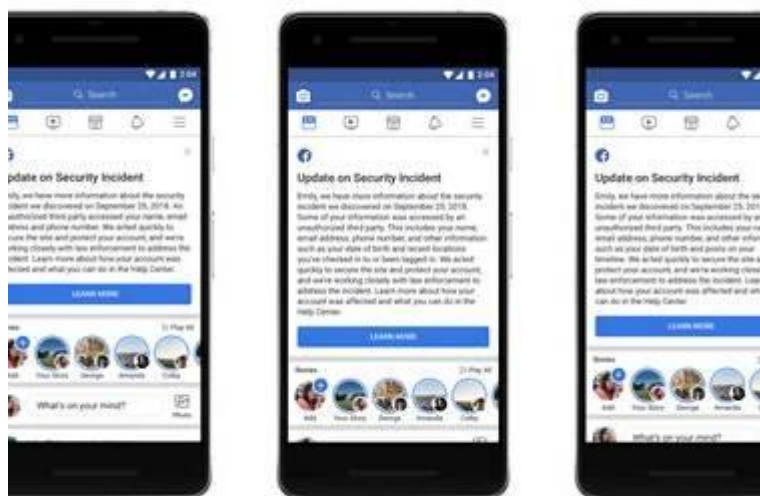


Figura 1 – Notificação de segurança exibida pelo Facebook após o incidente de 2018

Fonte: META. Security Update. 2018.

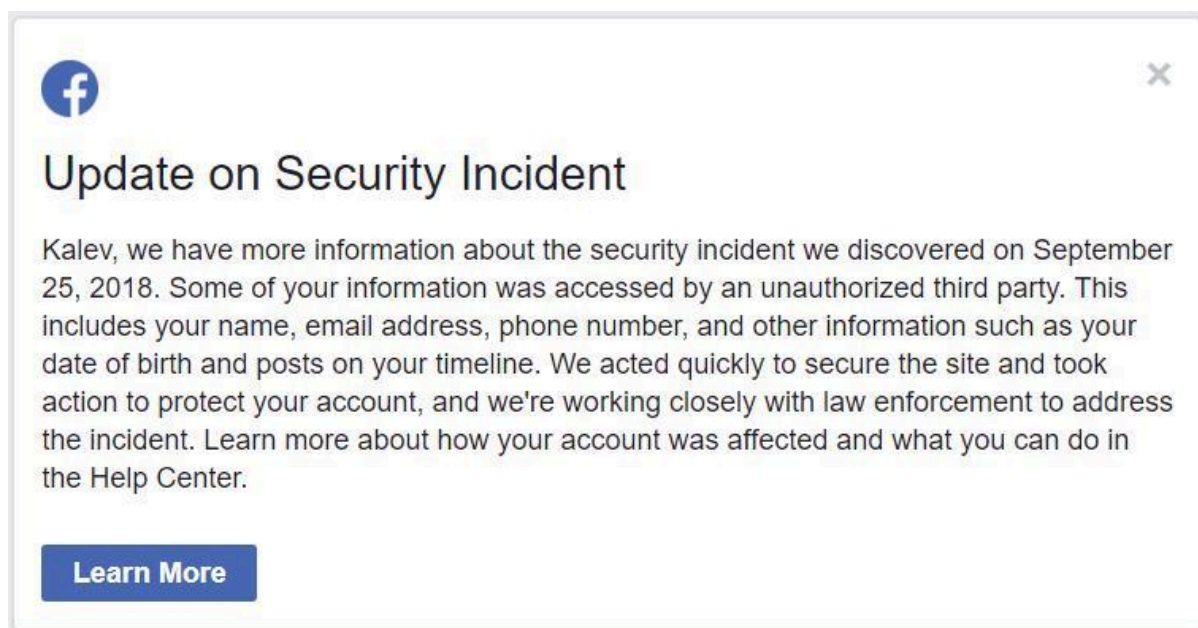


Figura 2 – Mensagem personalizada enviada pelo Facebook aos usuários afetados pelo incidente

Fonte: META. An Update on the Security Issue. 2018.

5. RECOMENDAÇÃO DE SEGURANÇA – AMEAÇA 1

5.1 Contramedidas para Broken Access Control

Para reduzir os riscos relacionados à vulnerabilidade Broken Access Control (Controle de Acesso Quebrado), deve-se implementar diferentes mecanismos de segurança, capazes de garantir que cada usuário tenha acesso somente aos recursos e funcionalidades necessários para as suas atividades. O controle de acesso deve ser aplicado principalmente no lado do servidor, pois pode não ser suficiente apenas ocultar páginas ou funcionalidades na interface da aplicação.

Uma das principais recomendações é adotar o princípio do menor privilégio, garantindo que cada usuário possua somente as permissões necessárias para realizar suas atividades. Dessa maneira, um funcionário que utiliza o sistema para consultar informações, por exemplo, não deve possuir permissões para excluir ou alterar esses mesmos dados. A separação adequada dos privilégios reduz o impacto caso uma conta seja comprometida.

Outra medida importante é realizar a validação das permissões em todas as requisições. O sistema deve verificar, no servidor, se o usuário autenticado realmente possui autorização para executar determinada operação ou acessar determinado recurso. Não se deve confiar apenas nas informações enviadas pelo navegador ou em elementos da interface.

A XPTO também deve implementar controles para evitar que um usuário consiga acessar diretamente recursos pertencentes a outro usuário. Por exemplo, se uma aplicação utilizar identificadores de usuários ou registros em suas requisições, o sistema deve verificar se o

usuário autenticado possui autorização para acessar aquele identificador. Dessa forma, simplesmente alterar um código ou identificador na URL ou na requisição não deverá permitir o acesso aos dados de outra pessoa.

Outra recomendação é utilizar o controle de acesso baseado em funções, conhecido como RBAC (Role-Based Access Control), quando esse modelo for adequado à aplicação. Nesse mecanismo, os usuários são associados a funções ou papéis, como administrador, gerente e usuário comum, e cada função possui um conjunto específico de permissões. Essa organização facilita a administração dos privilégios, reduzindo a possibilidade de que permissões sejam concedidas de maneira inadequada.

Também é importante realizar testes de segurança e testes de autorização durante o desenvolvimento da aplicação. A equipe responsável deve verificar se usuários com diferentes níveis de privilégio conseguem acessar somente aquilo que está autorizado para cada perfil.

A empresa deve ainda manter registros e monitoramento das atividades relacionadas ao controle de acesso. Tentativas repetidas de acessar recursos não autorizados, alterações incomuns de permissões ou até comportamentos incompatíveis com o perfil do usuário podem indicar uma tentativa de exploração. O monitoramento permite identificar comportamentos suspeitos e possibilita uma resposta mais rápida a possíveis incidentes.

Além disso, é recomendável realizar revisões periódicas das permissões dos usuários. Funcionários podem mudar de função, deixar a empresa ou passar a desempenhar atividades que exigem diferentes níveis de acesso. Por esse motivo, as permissões devem ser revisadas e atualizadas regularmente, evitando que usuários mantenham privilégios que não são mais necessários.

Por fim, a XPTO deve adotar uma abordagem de defesa em profundidade, utilizando diferentes mecanismos de proteção simultaneamente. Autenticação, autorização, validação no servidor, princípio do menor privilégio, testes de segurança, monitoramento e revisão periódica das permissões devem trabalhar em conjunto, visando uma proteção total, pois em um caso de uma camada de segurança ser violada, as outras medidas poderão contribuir para impedir ou limitar o comprometimento do sistema.

6. AMEAÇA 2 – INJECTION

6.1 Tipo de ameaça – A03:2021 Injection

A Injection (Injeção) é uma categoria de vulnerabilidade que ocorre quando uma aplicação envia dados fornecidos pelo usuário para um interpretador ou componente de processamento sem realizar a validação e o tratamento adequado dessas informações. Como consequência, um atacante pode conseguir inserir comandos ou informações maliciosas que sejam interpretadas pelo sistema de maneira diferente daquela prevista pelos desenvolvedores.

Esse tipo de vulnerabilidade pode ocorrer em diferentes tecnologias e componentes de uma aplicação. Entre os exemplos mais conhecidos está a SQL Injection, na qual comandos

maliciosos são inseridos em determinadas consultas destinadas a um certo banco de dados. Também existem outros tipos de injeção, como comandos de sistema operacional, consultas NoSQL e outras formas de manipulação de comandos ou interpretadores.

Um exemplo simples seria uma aplicação possuir uma tela de login que recebe um nome de usuário e uma senha e utiliza diretamente essas informações para montar uma consulta ao banco de dados. Caso os dados recebidos não sejam tratados adequadamente, um atacante pode tentar inserir uma entrada especialmente criada para alterar o comportamento da consulta original. Em uma aplicação vulnerável, isso pode permitir que o atacante contorne mecanismos de autenticação ou obtenha acesso indevido a informações.

A gravidade desse tipo de ameaça está principalmente no fato de que a entrada fornecida pelo usuário pode deixar de ser tratada como dados e passar a ser interpretada como parte de um comando. Dessa maneira, uma falha de validação pode permitir que um atacante influencie a execução de operações que deveriam ser controladas exclusivamente pela aplicação.

6.2 Possíveis impactos para a empresa XPTO

Uma vulnerabilidade de Injection pode causar diversos impactos para a empresa XPTO, dependendo do sistema afetado e dos privilégios disponíveis para o componente vulnerável.

Um dos principais riscos é o acesso não autorizado a informações armazenadas nos bancos de dados. Caso uma aplicação seja vulnerável a SQL Injection, por exemplo, um atacante pode conseguir manipular consultas e acessar dados que deveriam permanecer protegidos. Informações de clientes, dados financeiros, credenciais, registros internos e outras informações corporativas podem ficar expostas.

Outro possível impacto é a alteração ou exclusão de informações. Caso o atacante consiga influenciar comandos executados pelo banco de dados, poderá tentar modificar registros ou provocar a perda de dados. Em uma empresa, esse tipo de comprometimento pode afetar diretamente a operação de sistemas e, conseqüentemente, a confiabilidade das informações armazenadas.

A vulnerabilidade também pode contribuir para o bypass de mecanismos de autenticação, permitindo que um atacante consiga acessar determinadas áreas do sistema sem possuir credenciais legítimas. Se a aplicação possuir usuários com diferentes níveis de privilégio, uma exploração bem-sucedida pode aumentar ainda mais os impactos do incidente.

Além dos prejuízos técnicos, a exploração de uma vulnerabilidade de Injection pode gerar consequências financeiras e administrativas para a XPTO. O vazamento de informações pode provocar perda de confiança de clientes e parceiros, danos à reputação da empresa e possíveis consequências relacionadas à proteção de dados.

Portanto, a XPTO deve tratar as entradas fornecidas pelos usuários como dados potencialmente não confiáveis e implementar mecanismos adequados de validação, parametrização e proteção. A adoção dessas práticas reduz a possibilidade de que informações fornecidas por um usuário sejam interpretadas indevidamente como comandos pelo sistema.

7. EXEMPLO REAL – AMEAÇA 2

7.1 Caso TalkTalk – Ataque de SQL Injection

Um exemplo real de exploração da ameaça Injection ocorreu em outubro de 2015 contra a empresa britânica Talk Talk, provedora de serviços de telecomunicações. O ataque explora uma vulnerabilidade do tipo SQL Injection presente em páginas do site da empresa e permitiu que os invasores acessassem a base de dados que armazena informações pessoais de clientes. O caso foi posteriormente investigado pelo Information Commissioner's Office (ICO), órgão responsável pela proteção de dados no Reino Unido.

De acordo com a investigação do ICO, os invasores exploraram vulnerabilidades existentes em três páginas da internet operadas pela TalkTalk. Por meio dessas vulnerabilidades, foi possível obter acesso à base de dados subjacente, que armazenava informações de clientes, como nomes, endereços, datas de nascimento, números de telefone, endereços de e-mail e informações financeiras.

O ataque foi identificado pela empresa em 21 de outubro de 2015, após a TalkTalk perceber que sua rede estava funcionando de maneira mais lenta que o normal. A empresa identificou que se tratava de um ataque de SQL Injection e retirou seus sites do ar temporariamente. No dia seguinte, a empresa comunicou o incidente ao ICO e iniciou o processo de investigação e comunicação aos clientes.

7.2 Como ocorreu o comprometimento

O SQL Injection ocorre quando informações fornecidas pelo usuário são utilizadas de maneira insegura na construção de consultas ao banco de dados. Em vez de serem tratadas exclusivamente como dados, determinadas entradas podem alterar a lógica da consulta e fazer com que o banco de dados execute operações que não estavam previstas pelos desenvolvedores. A OWASP destaca que uma exploração bem-sucedida pode permitir a leitura, alteração ou exclusão de dados armazenados no banco de dados.

No caso da TalkTalk, os invasores utilizaram uma vulnerabilidade existente nas páginas da empresa para alcançar o banco de dados. A investigação do ICO identificou ainda que o software do banco de dados estava desatualizado e possuía uma vulnerabilidade em que a última correção disponível havia mais de três anos. Essa falha adicional permitia contornar determinadas restrições de acesso existentes no banco de dados.

Outro ponto relevante identificado pela investigação foi que a empresa já havia sofrido dois ataques anteriores de SQL Injection, em julho e setembro de 2015. Entretanto, esses incidentes não foram identificados adequadamente devido à ausência de monitoramento apropriado das páginas vulneráveis.

Esse aspecto demonstra a importância de combinar medidas preventivas com mecanismos de monitoramento. Mesmo que exista uma vulnerabilidade, a identificação rápida de tentativas de exploração pode ajudar uma organização a reduzir o impacto de um ataque.

7.3 Impactos do ataque

O incidente atingiu uma quantidade significativa de clientes da TalkTalk. Segundo o ICO, os dados pessoais de 156.959 clientes foram acessados pelos atacantes. Entre as informações potencialmente comprometidas estavam nomes, endereços, datas de nascimento, números de telefone e endereços de e-mail. Além disso, informações bancárias, como números de contas e códigos bancários, foram acessadas em 15.656 casos.

O comprometimento dessas informações representou um risco significativo para os clientes, principalmente devido à possibilidade de utilização dos dados em tentativas de fraude, golpes e roubo de identidade. O incidente também trouxe consequências para a própria TalkTalk, incluindo impactos financeiros, operacionais e reputacionais.

Após a investigação, o ICO concluiu que a empresa não havia adotado medidas técnicas e organizacionais adequadas para proteger os dados pessoais sob sua responsabilidade. Como consequência, a TalkTalk recebeu uma multa de £400.000, que na época foi considerada pelo ICO como sua maior multa aplicada até então.

7.4 Relação do caso com a empresa XPTO

O caso da TalkTalk demonstra como uma vulnerabilidade de Injection pode ultrapassar os limites de uma falha localizada em uma aplicação e resultar no comprometimento de informações armazenadas em bancos de dados.

Em um cenário semelhante na empresa XPTO, uma aplicação que utilize informações fornecidas pelos usuários para construir consultas ao banco de dados sem os mecanismos de proteção adequados poderia permitir a manipulação dessas consultas feita por um atacante. Dependendo das permissões disponíveis para a aplicação, isso poderia resultar no acesso, alteração ou até mesmo na exclusão de informações corporativas e dados de clientes.

O incidente também demonstra que a segurança não depende exclusivamente da existência de uma ferramenta de proteção. A TalkTalk possuía vulnerabilidades conhecidas e não corrigidas, além de falhas no monitoramento que dificultaram a identificação de ataques anteriores. Dessa forma, a XPTO deve adotar uma abordagem preventiva, envolvendo o desenvolvimento seguro, a atualização dos componentes, testes de segurança, monitoramento e aplicação do princípio do menor privilégio.

O caso da TalkTalk pode, portanto, ser utilizado como um exemplo concreto para demonstrar à empresa XPTO que vulnerabilidades de SQL Injection são capazes de gerar consequências significativas quando não são identificadas e corrigidas adequadamente.

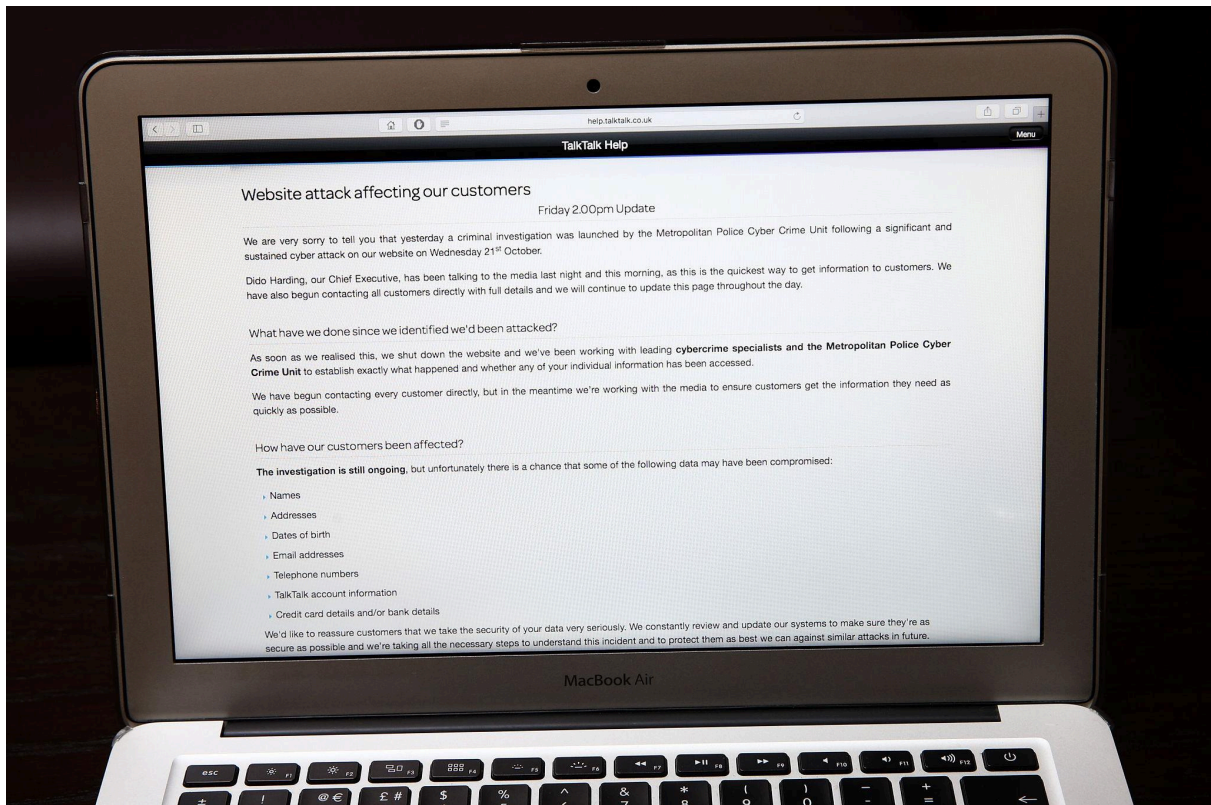


Figura 3 – Comunicado da TalkTalk sobre o ataque cibernético ocorrido em outubro de 2015.

Fonte: TALK TALK. *Cyber Attack update – Friday November 6th 2015*. 2015.

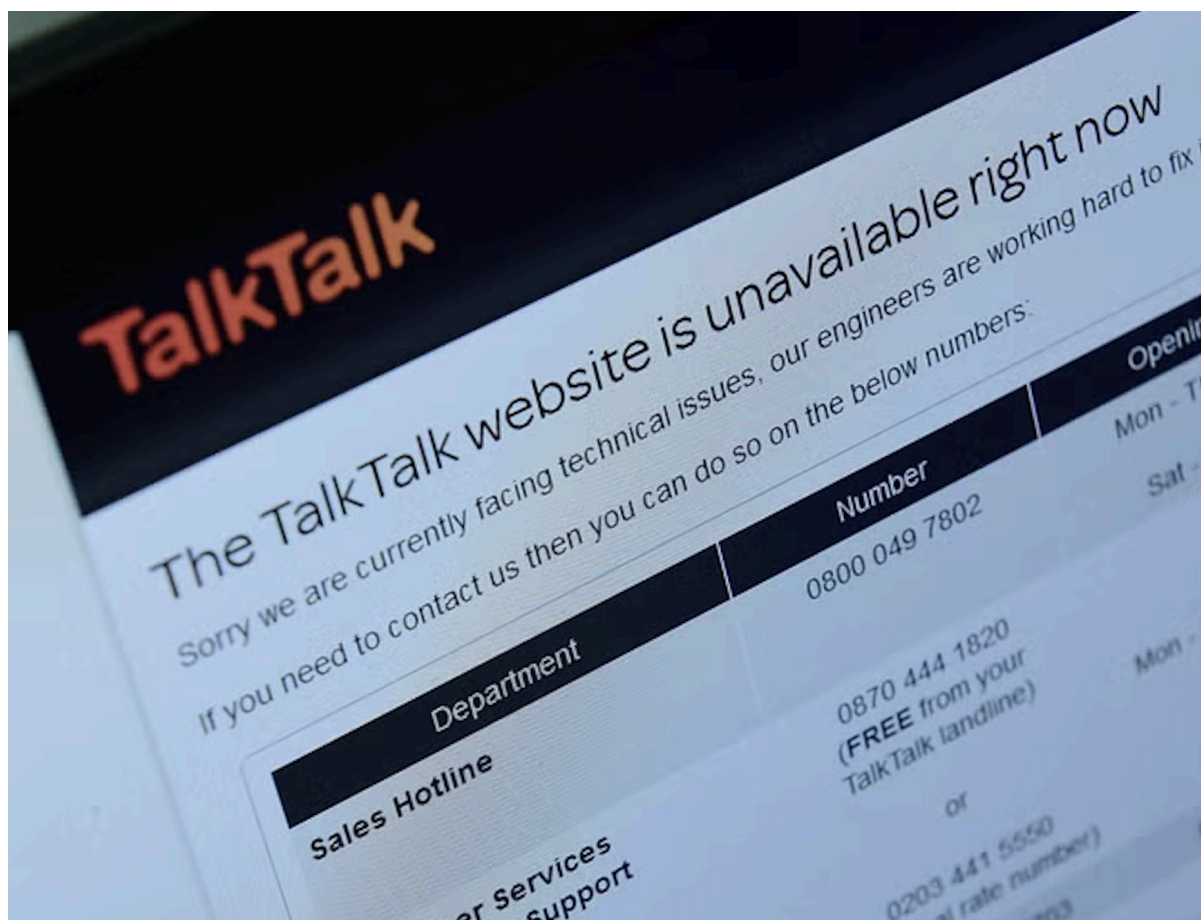


Figura 4 – Sede da TalkTalk após o ataque cibernético ocorrido em outubro de 2015.

Fonte: WIRED. *Hack Brief: TalkTalk Now Says Hackers Only Hit 4 Percent of Its Users*. 2015.

8. RECOMENDAÇÃO DE SEGURANÇA – AMEAÇA 2

8.1 Contramedidas para Injection

Para reduzir os riscos relacionados à vulnerabilidade Injection, especialmente aos ataques de SQL Injection, deve-se adotar medidas de segurança durante todo o ciclo de desenvolvimento de seus sistemas. O principal objetivo é impedir que as informações fornecidas pelos usuários sejam interpretadas como comandos pelo banco de dados ou por outros componentes da aplicação.

Uma das principais medidas de proteção é utilizar consultas parametrizadas (Prepared Statements). Nesse mecanismo, os comandos SQL e os valores fornecidos pelo usuário são tratados separadamente. Dessa forma, mesmo que um usuário tente inserir caracteres ou comandos maliciosos, essas informações serão interpretadas como dados e não como parte da instrução SQL. Essa prática reduz significativamente a possibilidade de exploração de SQL Injection.

Outra medida importante é realizar a validação das entradas fornecidas pelos usuários. A aplicação deve verificar se os dados recebidos estão de acordo com o formato esperado antes de processá-los. Por exemplo, um campo destinado a receber uma idade deve aceitar apenas valores compatíveis com esse tipo de informação. Entretanto, a validação de entrada não deve ser utilizada como única proteção contra Injection, devendo ser combinada com consultas parametrizadas e outras medidas de segurança.

Também deve-se evitar a construção de consultas SQL por meio da concatenação direta de informações fornecidas pelo usuário. Quando dados externos são inseridos diretamente em uma instrução SQL, aumenta-se o risco de que um atacante consiga modificar o comportamento da consulta. Portanto, os dados recebidos pela aplicação devem ser tratados de maneira adequada antes de serem utilizados em operações com o banco de dados.

Outra recomendação é utilizar contas de banco de dados com o menor nível de privilégio possível. A aplicação não deve utilizar uma conta com permissões administrativas completas quando não houver necessidade. Por exemplo, se determinado sistema apenas precisa consultar informações, sua conta de banco de dados não deveria possuir permissões para excluir ou modificar todas as tabelas. Essa medida reduz o impacto caso uma vulnerabilidade seja explorada.

A empresa também deve manter seus bancos de dados, sistemas operacionais, frameworks e bibliotecas atualizados. No caso analisado da TalkTalk, a investigação identificou a existência de software de banco de dados desatualizado e uma vulnerabilidade para a qual já havia uma correção disponível. Esse exemplo demonstra que a aplicação de atualizações e correções de segurança é uma parte importante da proteção contra ataques.

Outra medida fundamental é realizar testes de segurança durante o desenvolvimento da aplicação. A equipe pode utilizar testes automatizados, análise de código e testes de segurança para identificar pontos em que entradas externas possam ser utilizadas de maneira insegura. Os testes devem considerar diferentes tipos de entradas e verificar se a aplicação mantém o comportamento esperado diante de informações maliciosas.

A XPTO também deve implementar monitoramento e registro de atividades suspeitas. Tentativas repetidas de inserir informações incomuns em campos de entrada, erros inesperados do banco de dados ou grande quantidade de requisições malformadas podem indicar uma tentativa de exploração. O registro dessas atividades pode ajudar a equipe de segurança a identificar ataques e responder rapidamente a possíveis incidentes.

Além disso, recomenda-se realizar revisões de código com foco em segurança. Durante essas revisões, os desenvolvedores devem verificar se as aplicações utilizam consultas parametrizadas, se as entradas são tratadas adequadamente e se existem pontos em que dados fornecidos pelo usuário são utilizados diretamente em comandos ou consultas.

Por fim, a XPTO deve adotar uma estratégia de defesa em profundidade, combinando diferentes mecanismos de proteção. A utilização de consultas parametrizadas, validação de entradas, menor privilégio, atualização dos componentes, testes de segurança, revisão de código e monitoramento permite criar várias camadas de proteção. Dessa maneira, caso

uma medida apresente alguma falha, outras poderão contribuir para impedir ou limitar o impacto de uma possível exploração.

8.2 Aplicação das recomendações na empresa XPTO

Para a empresa XPTO, essas medidas devem fazer parte do processo de desenvolvimento e manutenção dos sistemas, e não serem aplicadas somente após a ocorrência de um incidente. Os desenvolvedores devem utilizar práticas de programação segura desde a criação da aplicação, enquanto a equipe responsável pela infraestrutura deve manter bancos de dados e demais componentes atualizados e monitorados.

Também é importante que a XPTO estabeleça procedimentos para identificar e corrigir rapidamente vulnerabilidades encontradas durante testes. A realização periódica de avaliações de segurança ajuda a identificar falhas antes que sejam descobertas e exploradas por atacantes.

Dessa forma, a combinação de controles preventivos, mecanismos de detecção e procedimentos de resposta permite que a empresa reduza consideravelmente os riscos associados à Injection. A adoção dessas práticas contribui para proteger os dados, preservando a disponibilidade dos sistemas e reduzindo os impactos financeiros e reputacionais que poderiam resultar de um ataque bem-sucedido.

9. Conclusão

A realização deste trabalho permitiu compreender melhor a importância do desenvolvimento seguro de sistemas e a necessidade de considerar a segurança desde as primeiras etapas de criação de uma aplicação. A utilização do modelo OWASP Top 10 possibilitou identificar diferentes categorias de riscos que podem comprometer aplicações e informações, além de demonstrar a importância de conhecer essas ameaças para que medidas preventivas possam ser adotadas pelas organizações.

Durante o desenvolvimento do trabalho, foram analisadas duas ameaças do modelo OWASP: Broken Access Control (Controle de Acesso Quebrado) e Injection (Injeção). A análise do Broken Access Control demonstrou como falhas na implementação de permissões podem permitir que usuários acessem informações ou funcionalidades que não deveriam estar disponíveis para eles. O caso do Facebook apresentado no trabalho mostrou como uma vulnerabilidade relacionada ao controle de acesso e à geração de tokens poderia resultar no comprometimento de milhões de contas.

Em relação à Injection, foi possível compreender como as entradas fornecidas pelos usuários podem ser utilizadas de maneira maliciosa quando não são devidamente tratadas pela aplicação. O caso da TalkTalk demonstrou que uma vulnerabilidade de SQL Injection pode permitir o acesso indevido a informações armazenadas em bancos de dados e gerar consequências significativas para uma organização, incluindo impactos financeiros, operacionais e reputacionais.

A partir dos dois casos analisados, também foi possível perceber que a segurança de uma aplicação não depende de apenas uma medida de proteção. É necessário utilizar diferentes

mecanismos de segurança de forma conjunta, como controle adequado de permissões, princípio do menor privilégio, validação de entradas, consultas parametrizadas, atualização de componentes, testes de segurança, monitoramento e revisão periódica dos sistemas.

Uma das principais percepções obtidas durante a realização do trabalho foi que muitas vulnerabilidades podem estar relacionadas a falhas que se parecem simples ou pequenas dentro de um sistema. Entretanto, quando exploradas por um atacante, essas falhas podem gerar consequências de grande proporção. Por esse motivo, as organizações precisam adotar uma postura preventiva e incorporar práticas de segurança ao longo de todo o ciclo de desenvolvimento de seus sistemas.

Em relação ao desenvolvimento do trabalho, a pesquisa dos exemplos reais foi uma etapa importante para compreender como os conceitos estudados podem ser observados em situações concretas. A análise desses incidentes também facilitou a compreensão dos possíveis impactos que uma vulnerabilidade pode causar em uma organização. Como dificuldade, destaca-se a necessidade de relacionar os conceitos teóricos do modelo OWASP com acontecimentos reais, identificando medidas de segurança adequadas para cada tipo de ameaça.

Conclui-se, portanto, que a adoção de práticas de desenvolvimento seguro é fundamental para reduzir os riscos de comprometimento dos sistemas da empresa XPTO. O conhecimento das ameaças apresentadas pelo OWASP Top 10, aliado à implementação de medidas preventivas e ao monitoramento contínuo, permite que as organizações estejam mais preparadas para identificar, prevenir e responder a possíveis ataques. Dessa forma, investir em segurança desde o desenvolvimento dos sistemas contribui não apenas para a proteção das informações, mas também para a continuidade das operações e para a confiança de clientes e parceiros.

10. Referências Bibliográficas

INFORMATION COMMISSIONER'S OFFICE (ICO). TalkTalk cyber attack – how the ICO's investigation unfolded. [S. l.], [s. d.]. Disponível em: <https://ico.org.uk/about-the-ico/media-centre/talktalk-cyber-attack-how-the-ico-investigation-unfolded/>. Acesso em: 21 ago. 2026.

META. Security update. [S. l.], 28 set. 2018. Disponível em: <https://about.fb.com/news/2018/09/security-update/>. Acesso em: 21 ago. 2026.

META. Update on security issue. [S. l.], 12 out. 2018. Disponível em: <https://about.fb.com/news/2018/10/update-on-security-issue/>. Acesso em: 21 ago. 2026.

OWASP FOUNDATION. A01:2021 – Broken Access Control. [S. l.], 2021. Disponível em: https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/. Acesso em: 21 ago. 2026.

OWASP FOUNDATION. OWASP Developer Guide: OWASP Top Ten. [S. l.], [s. d.]. Disponível em: <https://devguide.owasp.org/pt-br/02-foundations/05-top-ten/>. Acesso em: 21 ago. 2026.

OWASP FOUNDATION. OWASP Top 10:2021. [S. l.], 2021. Disponível em:
<https://owasp.org/Top10/2021/>. Acesso em: 21 ago. 2026.

OWASP FOUNDATION. SQL Injection. [S. l.], [s. d.]. Disponível em:
https://owasp.org/www-community/attacks/SQL_Injection. Acesso em: 21 ago. 2026.